

EMo Learners

PROFESSIONAL QUESTION BANK WITH FULL SOLUTIONS

Analysis and Design of Algorithms (ADA)

Subjective, Comparative, Derivation, and Tracing Solutions

Subject: Analysis & Design of Algorithms (ADA)

Scope: Units 1 & 2 (Detailed Textbook Answers)

Prep Quality: Verified & Curated

Branding Highlight: EMo Learners Exclusive

UNIT 1: ALGORITHMS, DESIGNING, AND ANALYZING

Q1. [Short Answer] What is an algorithm? Explain the various characteristics of a good algorithm.

Answer:

An algorithm is a well-defined, step-by-step computational procedure that takes some value (or set of values) as input and produces some value (or set of values) as output to solve a specific mathematical or computational problem. It serves as an abstract recipe for solving a problem, independent of specific programming languages or execution environments.

A good and correct algorithm must exhibit the following six key characteristics:

- **Input:** An algorithm must have zero or more well-defined inputs, supplied from an external source.
- **Output:** It must produce at least one well-defined output value that solves the target problem.
- **Definiteness (Unambiguous):** Each step of the algorithm must be clear, precise, and completely unambiguous. There should be only one interpretation of each instruction.
- **Finiteness:** The algorithm must always terminate after a finite number of execution steps, avoiding infinite loops.
- **Feasibility (Effectiveness):** Every step must be basic enough to be carried out in practice, meaning it is realizable with paper and pencil using finite resources.
- **Language Independence:** The steps must be purely logical and generic, allowing direct translation into any programming language (C, Python, Java, etc.).

Q2. [Short Answer] Define Asymptotic Notations. Explain Big-Oh (O), Omega (Ω), and Theta (Θ) notations with suitable examples.

Answer:

Asymptotic Notations are mathematical tools used to describe and analyze the running time or space complexity of an algorithm in terms of the input size n as n grows arbitrarily large (approaching infinity). They provide a way to establish boundaries on growth rates.

The three primary asymptotic notations are defined and exemplified below:

1. Big-Oh Notation (O) — Asymptotic Upper Bound

Used to describe the worst-case scenario. It guarantees that an algorithm will never take longer than this upper limit.

- Formal Definition: $T(n) = O(f(n))$ if there exist positive constants c and n_0 such that:
 $0 \leq T(n) \leq c \cdot f(n)$ for all $n \geq n_0$.
- Example: Let $T(n) = 3n + 2$. We can claim $T(n) = O(n)$ because $3n + 2 \leq 4n$ for all $n \geq 2$. Here, our constants are $c = 4$ and $n_0 = 2$.

2. Big-Omega Notation (Ω) — Asymptotic Lower Bound

Used to describe the best-case scenario. It guarantees that an algorithm will take at least this much time.

- Formal Definition: $T(n) = \Omega(f(n))$ if there exist positive constants c and n_0 such that:
 $0 \leq c \cdot f(n) \leq T(n)$ for all $n \geq n_0$.
- Example: Let $T(n) = 3n + 2$. We can claim $T(n) = \Omega(n)$ because $3n + 2 \geq 3n$ for all $n \geq 1$. Here, our constants are $c = 3$ and $n_0 = 1$.

3. Theta Notation (Θ) — Asymptotic Tight Bound

Used to describe the average-case running time. It bounds the function from both above and below.

- Formal Definition: $T(n) = \Theta(f(n))$ if there exist positive constants c_1 , c_2 , and n_0 such that:
 $0 \leq c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$ for all $n \geq n_0$.
- Example: Let $T(n) = 3n + 2$. We can claim $T(n) = \Theta(n)$ because we can sandwich it: $3n \leq 3n + 2 \leq 4n$ for all $n \geq 2$. Here, $c_1 = 3$, $c_2 = 4$, and $n_0 = 2$.

Q3. [Short Answer] Compare Quick Sort and Merge Sort on the basis of their performance, memory usage, and time complexity.

Answer:

Quick Sort and Merge Sort are two of the most popular divide-and-conquer sorting algorithms. A comprehensive, spacious comparison is tabulated below:

Comparison Parameter	Quick Sort	Merge Sort
Design Paradigm	Divide-and-Conquer. Relies on partitioning the array around a pivot element.	Divide-and-Conquer. Splits the array in halves recursively and merges them.
Time Complexity	Best/Average: $O(n \log n)$ Worst-Case: $O(n^2)$ (when partition is highly skewed)	Best/Average/Worst: $O(n \log n)$ (highly consistent, independent of input order)

Auxiliary Space	$O(\log n)$ (very efficient, in-place sorting relying only on recursion stack)	$O(n)$ (requires an extra temporary array of size n to merge elements)
Stability	Unstable (may swap equal elements and change their relative original positions)	Stable (guarantees relative order of equal elements is preserved)
Data Suitability	Excellent for arrays due to high cache locality and in-place sorting.	Highly optimal for linked lists and massive external sorting files.

Q4. [Long Answer] Discuss Strassen's matrix multiplication algorithm. Derive its time complexity and explain how it improves upon conventional matrix multiplication.

Answer:

Conventional matrix multiplication of two $n \times n$ matrices requires splitting each into four submatrices of size $n/2 \times n/2$, which leads to 8 recursive multiplications and 4 additions. This yields the recurrence relation:

$T(n) = 8T(n/2) + \Theta(n^2)$. By Master Theorem, this solves to $O(n^3)$.

Strassen's Breakthrough: Volker Strassen showed that we can multiply matrices using only 7 multiplications and 18 additions/subtractions, improving asymptotic performance.

The 7 Sub-Multiplication Products (P_1 to P_7):

Let matrices A and B be partitioned into $n/2 \times n/2$ submatrices. The 7 strategic products are defined as:

- $P_1 = A_{11} \cdot (B_{12} - B_{22})$
- $P_2 = (A_{11} + A_{12}) \cdot B_{22}$
- $P_3 = (A_{21} + A_{22}) \cdot B_{11}$
- $P_4 = A_{22} \cdot (B_{21} - B_{11})$
- $P_5 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$
- $P_6 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$
- $P_7 = (A_{11} - A_{21}) \cdot (B_{11} + B_{12})$

Combining Products for Result Matrix C :

- $C_{11} = P_5 + P_4 - P_2 + P_6$
- $C_{12} = P_1 + P_2$
- $C_{21} = P_3 + P_4$
- $C_{22} = P_5 + P_1 - P_3 - P_7$

Derivation of Time Complexity:

Since there are 7 recursive multiplications and additions take quadratic time for merging, the recurrence relation is:

$T(n) = 7T(n/2) + \Theta(n^2)$

Using the Master Theorem of the form $T(n) = aT(n/b) + f(n)$:

- $a = 7$, $b = 2$, and $f(n) = \Theta(n^2)$
- Compare $f(n)$ with $n^{\log_b a} = n^{\log_2 7} \approx n^{2.81}$
- Since $2 < 2.81$, this falls under Case 1 of Master Theorem.
- Therefore, the complexity is $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$.

How it Improves: For large-scale matrices (typically $n > 128$), Strassen's algorithm drastically reduces CPU multiplication operations. Standard matrix multiplication grows at a rate of n^3 , whereas Strassen's grows at $n^{2.81}$, which saves billions of operations as the size scales.

Q5. [Long Answer] Explain the Heap sort technique. Trace the steps to sort the following array using heap sort: 81, 39, 10, 36, 45, 15, 55, 23, 91, 88, 12.

Answer:

Heap Sort is a comparison-based sorting technique based on a Binary Heap data structure. It is an in-place algorithm that has a guaranteed running time of $O(n \log n)$.

Phase 1: Build Max-Heap (Step-by-Step heapification):

We begin heapifying starting from the last non-leaf node index $\lfloor (11-2)/2 \rfloor = 4$ (element 45) up to the root index 0:

- Heapify at Index 4 (Value 45): Children are at index 9 (88) and index 10 (12). Since child 88 is greater than 45, we swap them.
Resulting Array State: [81, 39, 10, 36, 88, 15, 55, 23, 91, 45, 12]
- Heapify at Index 3 (Value 36): Children are at index 7 (23) and index 8 (91). Since child 91 is greater than 36, we swap them.
Resulting Array State: [81, 39, 10, 91, 88, 15, 55, 23, 36, 45, 12]
- Heapify at Index 2 (Value 10): Children are at index 5 (15) and index 6 (55). Since child 55 is greater than 10, we swap them.
Resulting Array State: [81, 39, 55, 91, 88, 15, 10, 23, 36, 45, 12]
- Heapify at Index 1 (Value 39): Children are at index 3 (91) and index 4 (88). Since child 91 is greater than 39, we swap them. No sub-heapify swap is needed as 39 is greater than its new children 23 and 36.
Resulting Array State: [81, 91, 55, 39, 88, 15, 10, 23, 36, 45, 12]
- Heapify at Index 0 (Value 81): Children are index 1 (91) and index 2 (55). Since 91 is greater, swap 81 and 91. Next, sub-heapify index 1 (value 81) with children index 3 (39) and index 4 (88). Swap 81 and 88.
Resulting Max-Heap State: [91, 88, 55, 39, 81, 15, 10, 23, 36, 45, 12]

Max-Heap Constructed successfully: [91, 88, 55, 39, 81, 15, 10, 23, 36, 45, 12]

Phase 2: Repeatedly Extract Max & Sort:

We swap the root element (index 0) with the last element of the active heap, reduce the heap size by 1, and heapify the root. We repeat this 10 times to sort the list:

- Step 1: Swap root 91 and last index 12. Heap size = 10. Heapify root 12.
Active Heap: [88, 81, 55, 39, 45, 15, 10, 23, 36, 12] | Sorted: [91]

- Step 2: Swap root 88 and last index 12. Heap size = 9. Heapify root 12.
Active Heap: [81, 45, 55, 39, 12, 15, 10, 23, 36] | Sorted: [88, 91]
- Step 3: Swap root 81 and last index 36. Heap size = 8. Heapify root 36.
Active Heap: [55, 45, 36, 39, 12, 15, 10, 23] | Sorted: [81, 88, 91]
- Step 4: Swap root 55 and last index 23. Heap size = 7. Heapify root 23.
Active Heap: [45, 39, 36, 23, 12, 15, 10] | Sorted: [55, 81, 88, 91]
- Step 5: Swap root 45 and last index 10. Heap size = 6. Heapify root 10.
Active Heap: [39, 23, 36, 10, 12, 15] | Sorted: [45, 55, 81, 88, 91]
- Step 6: Swap root 39 and last index 15. Heap size = 5. Heapify root 15.
Active Heap: [36, 23, 15, 10, 12] | Sorted: [39, 45, 55, 81, 88, 91]
- Step 7: Swap root 36 and last index 12. Heap size = 4. Heapify root 12.
Active Heap: [23, 12, 15, 10] | Sorted: [36, 39, 45, 55, 81, 88, 91]
- Step 8: Swap root 23 and last index 10. Heap size = 3. Heapify root 10.
Active Heap: [15, 12, 10] | Sorted: [23, 36, 39, 45, 55, 81, 88, 91]
- Step 9: Swap root 15 and last index 10. Heap size = 2. Heapify root 10.
Active Heap: [12, 10] | Sorted: [15, 23, 36, 39, 45, 55, 81, 88, 91]
- Step 10: Swap root 12 and last index 10. Heap size = 1. Active Heap stabilizes at root [10].
Sorted list: [10, 12, 15, 23, 36, 45, 55, 81, 88, 91]

Final Sorted Output: [10, 12, 15, 23, 36, 45, 55, 81, 88, 91]

Q6. [Long Answer] Design the Merge Sort algorithm. Apply it to sort a given list and discuss its best-case, average-case, and worst-case time complexities.

Answer:

Merge Sort is a classic divide-and-conquer sorting algorithm. It divides the input array into two halves, calls itself for the two halves, and then merges the two sorted halves using a helper function.

Algorithm Pseudocode:

MERGE-SORT(A, p, r):

1. **if** $p < r$
2. $q = \lfloor (p + r) / 2 \rfloor$
3. MERGE-SORT(A, p, q)
4. MERGE-SORT(A, q + 1, r)
5. MERGE(A, p, q, r)

MERGE(A, p, q, r):

1. $n1 = q - p + 1$
2. $n2 = r - q$
3. Let $L[1..n1+1]$ and $R[1..n2+1]$ be new arrays
4. **for** $i = 1$ **to** $n1$: $L[i] = A[p + i - 1]$

```
5. for j = 1 to n2: R[j] = A[q + j]
6. L[n1 + 1] = ∞, R[n2 + 1] = ∞
7. i = 1, j = 1
8. for k = p to r
9.     if L[i] ≤ R[j]
10.        A[k] = L[i]; i = i + 1
11.     else A[k] = R[j]; j = j + 1
```

Complexity Analysis:

The division step takes constant time, and merging two arrays of sizes $n/2$ takes $O(n)$ time. This leads to the recurrence relation:

$$T(n) = 2T(n/2) + O(n)$$

- Best-Case Time Complexity: $O(n \log n)$ — Even if the array is already fully sorted, the recursive splitting and linear merging are executed.
- Average-Case Time Complexity: $O(n \log n)$ — Occurs for arbitrary random arrays.
- Worst-Case Time Complexity: $O(n \log n)$ — Occurs when elements are reverse sorted. It is highly robust as it does not rely on pivot selection.

UNIT 2: STUDY OF GREEDY STRATEGY

Q1. [Short Answer] Describe the Greedy approach to algorithm designing. What are its primary characteristics?

Answer:

The Greedy Strategy is an algorithmic paradigm that builds up a solution piece-by-piece, always choosing the next piece that offers the most immediate and obvious benefit (locally optimal choice) with the hope that these choices lead to a globally optimal solution.

Primary Characteristics of a Greedy Algorithm:

- Greedy Choice Property: A globally optimal solution can be arrived at by making locally optimal (greedy) choices. The algorithm makes whatever choice seems best at the moment without looking ahead to future consequences.
- Optimal Substructure: A problem exhibits optimal substructure if an optimal solution to the entire problem contains optimal solutions to its subproblems.
- Irreversibility (No Backtracking): Once a decision is made in a greedy algorithm, it is completely permanent and cannot be altered or undone (unlike Backtracking or Dynamic Programming).

Q2. [Short Answer] Tabulate the differences between Kruskal's and Prim's algorithms for finding a minimum cost spanning tree.

Answer:

Both Kruskal's and Prim's are greedy algorithms used to construct a Minimum Spanning Tree (MST) of a connected weighted graph. Their core differences are tabulated below:

Parameter	Kruskal's Algorithm	Prim's Algorithm
MST Growth Style	Grows the Minimum Spanning Tree by selecting edges one-by-one based on sorting order.	Grows the Spanning Tree by connecting vertices one-by-one to a single root node.
Structure During Execution	Maintains a 'forest' (a collection of disconnected trees) that eventually merges.	Maintains a single connected tree at all times during the algorithm.
Optimal Data Structures	Disjoint Set Union (Union-Find) to detect cycles, and edge sorting.	Priority Queue (Min-Heap) and adjacency list representation.
Time Complexity	$O(E \log E)$ or $O(E \log V)$ (dominated by initial sorting of edges)	$O(E \log V)$ with binary heap; $O(E + V \log V)$ with Fibonacci heap
Best Suited For	Sparse graphs with relatively few edges (where $E \approx V$).	Dense graphs with a high number of edges (where $E \approx V^2$).

Q3. [Short Answer] What is the Knapsack problem? Briefly explain how the Fractional Knapsack problem is solved using the Greedy method.

Answer:

The Knapsack Problem is a classic optimization problem. Given a set of items, each with a weight w_i and a profit p_i , and a knapsack with maximum weight capacity W , the objective is to choose items to include in the knapsack to maximize the total profit without exceeding the capacity.

Fractional Knapsack Solution (Greedy Method):

In the Fractional version, we can take fractions of items (e.g., liquid, powder). The Greedy Strategy yields the mathematically optimal solution via these steps:

- Calculate Profit Density: For each item, compute the profit-to-weight ratio $r_i = p_i / w_i$.
- Sort Items: Sort all items in descending order of their ratio r_i .
- Fill Knapsack Greedily: Take as much of the item with the highest ratio as possible. If the knapsack capacity accommodates the entire item, pack it and subtract its weight from the capacity. If the capacity is less than the item's weight, take the exact fraction of the item that fills the remaining capacity and terminate.

Q4. [Long Answer] Write an algorithm for the Single Source Shortest Path (Dijkstra's Algorithm). Explain it with a suitable graph example and discuss its time complexity.

Answer:

Dijkstra's Algorithm solves the single-source shortest path problem on a directed or undirected graph with non-negative edge weights.

Algorithm Pseudocode:

1. Initialize $\text{dist}[v] = \infty$ for all vertices, $\text{dist}[\text{source}] = 0$.
2. Create a priority queue Q containing all vertices with key as $\text{dist}[v]$.
3. **while** Q is not empty:
4. $u = \text{Extract-Min}(Q)$
5. **for** each adjacent neighbor v of u :
6. **if** $\text{dist}[u] + \text{weight}(u, v) < \text{dist}[v]$:
7. $\text{dist}[v] = \text{dist}[u] + \text{weight}(u, v)$
8. $\text{Decrease-Key}(Q, v)$

Detailed Step-by-Step Distance Relaxation:

Let us solve for source vertex A on a 5-vertex graph with edges: A-B(4), A-C(2), B-C(1), B-D(5), C-D(8), C-E(10), D-E(2).

- Initialization: Set source distance $\text{dist}[A] = 0$ and all other vertex distances to ∞ .

Initial Distance Vector: [A: 0, B: ∞ , C: ∞ , D: ∞ , E: ∞]

- Step 1: Extract Source A ($\text{dist}[A] = 0$):

Relax adjacent neighbors of A:

- $\text{dist}[B] = \min(\infty, 0 + 4) = 4$

- $\text{dist}[C] = \min(\infty, 0 + 2) = 2$

Distance Vector Updates: [A: 0, B: 4, C: 2, D: ∞ , E: ∞]

- Step 2: Extract Vertex C ($\text{dist}[C] = 2$):

Relax adjacent neighbors of C:

- $\text{dist}[B] = \min(4, 2 + 1) = 3$ (Shorter path found!)

- $\text{dist}[D] = \min(\infty, 2 + 8) = 10$

- $\text{dist}[E] = \min(\infty, 2 + 10) = 12$

Distance Vector Updates: [A: 0, B: 3, C: 2, D: 10, E: 12]

- Step 3: Extract Vertex B ($\text{dist}[B] = 3$):

Relax adjacent neighbors of B:

- $\text{dist}[D] = \min(10, 3 + 5) = 8$ (Shorter path found!)

Distance Vector Updates: [A: 0, B: 3, C: 2, D: 8, E: 12]

- Step 4: Extract Vertex D ($\text{dist}[D] = 8$):

Relax adjacent neighbors of D:

- $\text{dist}[E] = \min(12, 8 + 2) = 10$ (Shorter path found!)

Distance Vector Updates: [A: 0, B: 3, C: 2, D: 8, E: 10]

- Step 5: Extract Vertex E ($\text{dist}[E] = 10$):
All vertices have successfully been extracted and processed.

Final Shortest Path Distances from A: A = 0, B = 3, C = 2, D = 8, E = 10.

Q5. [Long Answer] Apply both Kruskal's and Prim's algorithms to find the Minimum Spanning Tree for a given weighted graph. Compare their time complexities.

Answer:

Let us consider a weighted graph of 4 vertices: A, B, C, D, and 5 edges: (A,B):1, (B,D):2, (B,C):3, (A,C):4, (C,D):5.

Comparative Step-by-Step Edge Selection Trace:

We trace Kruskal's (which sorts all edges) and Prim's (which grows from a starting tree, chosen here as starting vertex A):

- Edge 1: (A, B) [weight 1]:
 - Kruskal's Action: Selected. Vertices A and B are in disconnected components, so they are merged into component {A, B}.
 - Prim's Action: Selected as the cheapest edge adjacent to the starting tree {A}. Visited tree grows to {A, B}. MST Cost = 1.
- Edge 2: (B, D) [weight 2]:
 - Kruskal's Action: Selected. Vertices B and D are in disconnected components, so they are merged. Component becomes {A, B, D}.
 - Prim's Action: Selected as the cheapest edge adjacent to the visited tree {A, B}. Visited tree grows to {A, B, D}. MST Cost = 1 + 2 = 3.
- Edge 3: (B, C) [weight 3]:
 - Kruskal's Action: Selected. Vertices B and C are in disconnected components, so they are merged. All vertices are now connected. Component = {A, B, C, D}.
 - Prim's Action: Selected as the cheapest edge connecting visited tree {A, B, D} to unvisited vertex C. Visited tree becomes {A, B, C, D}. MST Cost = 3 + 3 = 6.
- Edge 4: (A, C) [weight 4]:
 - Kruskal's Action: Rejected. Both A and C are already in the same component. Adding this would form a cycle.
 - Prim's Action: Ignored. Both A and C are already in the visited tree (forms cycle).
- Edge 5: (C, D) [weight 5]:
 - Kruskal's Action: Rejected. Forms a cycle.
 - Prim's Action: Ignored. Forms a cycle.

Final Spanning Tree Cost: Total Weight = 1 + 2 + 3 = 6 for both algorithms.

Q6. [Long Answer] Find the maximum profit using the Job Sequencing with Deadline algorithm. (Be prepared to solve a numerical matrix featuring Jobs, Profits, and Deadlines).**Answer:**

We consider a set of 5 jobs, each with an associated profit and deadline as defined in the matrix below:

Job ID	Profit	Deadline	Sorting Order (By Profit)
J ₁	100	2	1st (Highest)
J ₂	19	1	4th
J ₃	27	2	2nd
J ₄	25	1	3rd
J ₅	15	3	5th (Lowest)

Step-by-Step Greedy Allocation:

- Maximum Timeline: The maximum deadline is 3. We establish 3 slots: [0-1], [1-2], and [2-3].
- Job J₁: Has the highest profit (100). We assign it to its preferred slot [1-2] (empty). Timeline = [_, J₁, _].
- Job J₃: Second highest profit (27, deadline 2). Slot [1-2] is occupied. We look left and place J₃ in the empty slot [0-1]. Timeline = [J₃, J₁, _].
- Job J₄: Third highest profit (25, deadline 1). Preferred slot [0-1] is occupied by J₃. No earlier slots exist. J₄ is rejected.
- Job J₂: Fourth highest profit (19, deadline 1). Slot [0-1] is occupied. J₂ is rejected.
- Job J₅: Lowest profit (15, deadline 3). We assign it to its preferred empty slot [2-3]. Timeline = [J₃, J₁, J₅].

Final Scheduled Output Timeline:

Time Slot	Scheduled Job	Profit Added
[0 to 1]	J ₃ (Since slot [1-2] was occupied by J ₁ , J ₃ is placed in this empty slot)	27
[1 to 2]	J ₁ (Placed in the empty slot corresponding to its deadline of 2)	100
[2 to 3]	J ₅ (Placed in the empty slot corresponding to its deadline of 3)	15

Final Maximum Profit: 27 + 100 + 15 = 142.